

知能情報工学実験演習II

C++プログラミング

Part 2 : グループ演習

担当 : 平田, 山本

TA : 石丸, 磯部

グループ演習

- 1 グループ: 5～6 人
 - ・ グループ単位でプログラムを作成
- 「大富豪 (大貧民)」の思考ルーチンを作成
 - ・ 強いコンピュータプレイヤーを作る
- 講義資料, スライド, ソースファイルは moodle で提供

大富豪 (大貧民)

- 52 枚のカードとジョーカー 1 枚を使用
 - 強さは, **ジョーカー > 2 > A > K > Q > J > 10 > ... > 3** の順
 - ジョーカーは組のワイルドカードとしても使用可
- 親(リーダー)は好きなカード(の組)を場に出せる
- プレイヤは場に出ているカード(リード)よりも強いカードを出さなければならない
 - 組が出ていたら, 同じ枚数の組でなければならない
 - 出せない, または, 出したくない場合はパス
- 全員がパスをして一巡したら, 場のカードは流して, 最後にカード(リード)を出したプレイヤが新たな親(リーダー)となる
- 手札がなくなったプレイヤは上がり
 - 早く上がった方から順に順位がつく

大富豪 (大貧民) 演習でのルール

● 今回の演習で使うルール

- 2 やジョーカーがあがってもよい
- シークエンス(3, 4, 5 など)は不可
- 革命なし
- 搾取(大貧民から強いカードを大富豪が取り上げる)はなし
- 都落ちなし
- その他のローカルルールも考えない
- 次のゲームでは, 上がった順の逆順にターンが並ぶ. 前回の最下位が最初の親

サンプルプログラム

- `card.{h|cpp}`, `cardSet.{h|cpp}`
 - 基本的には `part1` と同様だが、**機能拡張している** ので必ずこちらを使うこと
- `dealer.{h|cpp}`, `daifugou.cpp`
 - ゲームの管理者, および, ゲームプログラムの本体
- **`player.{h|cpp}`**
 - プレイヤの基礎となるクラス(基底クラス)
- **`simpleplayer.{h|cpp}`, `defaultplayer.{h|cpp}`**
 - `Player` クラスを継承して作るプレイヤのサンプル
 - 関数など参考にしてよい
- `gamestatus.h`
 - ゲーム状態を表すデータ構造のクラス

サンプルプログラム

- thinkTA1.{h|o} (thinkTA1.o → libplayingcards.a)
 - 昔のTAが作成したプレイヤ，ソースコードは非公開
 - 練習試合で活用しよう
- コンパイルをサポートするための Makefile
 - コマンド make だけですべてコンパイルが可能
 - ファイルのタイムスタンプをみて，アップデートされたファイルに依存したものだけ再コンパイルする
 - make clean とすると *.o ファイルを削除し，すべてのコンパイルをやり直しできる

Dealer クラス

- ゲームを管理するディーラーのクラス
 - プレイヤとゲームの管理をする
 - カードの配布, 場に出されたカードが受理できるかどうかの判定などを行ない, ルールに従ってゲームを進行させる
 - ディーラーはプレイヤとは違う別物
- 参考のためソースコードを提供するが, 変更せずに使用すること

Player クラス

- プレイヤの基礎となるクラス（基底クラス）
 - 手持ちのカードとプレイヤ名
 - 重要なメンバ関数
 - `bool follow(GameStatus &, CardSet &);`
 - ・ カードを出す処理
 - `bool approve(GameStatus &);`
 - ・ ターンのプレイヤが場に何を出したかの確認
 - `void ready();`
 - ・ 新しいゲームが始まる合図
- 最小限のサンプルを与えている
 - サブクラスでこの3つの関数をオーバーライドする
- **Playerクラスを直接変更してはならない**

DefaultPlayer クラスとSimplePlayerクラス

- プレイヤは Player クラスを継承して作成
- 例として次の2つのクラスを提供
 - DefaultPlayer クラス
 - Player クラスを継承しただけのクラス
 - 自分では何も作成していない
 - follow ではランダムに1枚出す
 - SimplePlayer クラス
 - 出せるカードは必ず出す
 - 貪欲な戦略のプレイヤーの例
 - `void CardSet::sort(void)` を利用してソート

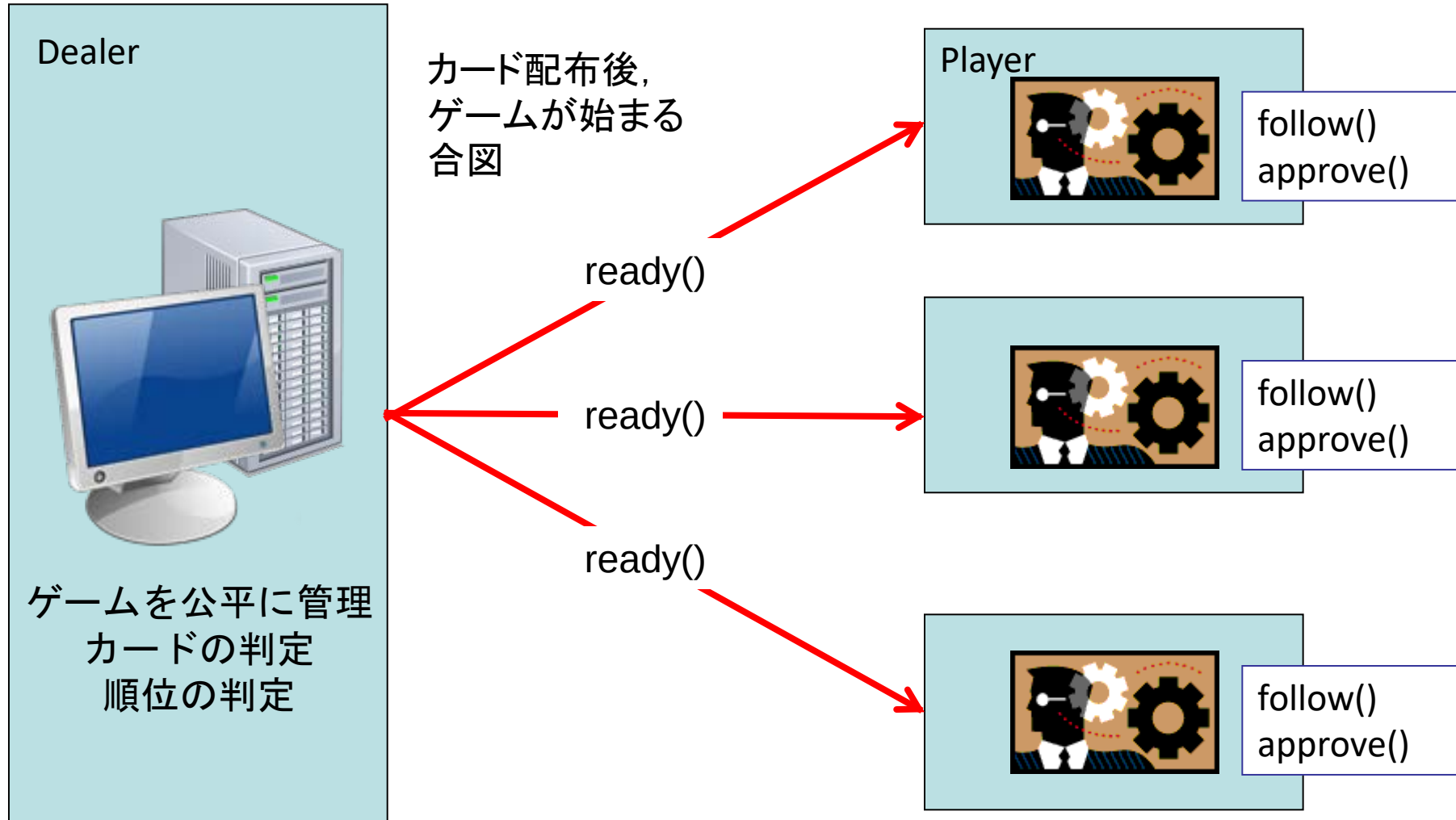
カードの判定(1)

- ゲームの進行, 管理
 - Dealer クラスが行うので考えなくて良い
 - 判定は Dealer::accept() が行う
 - 受理されれば, 場にカードを出して true
 - 受理されなければ, カードは手持ちに帰り false
- 複数枚組のチェックも実装されている
 - ジョーカーを含むペアも可

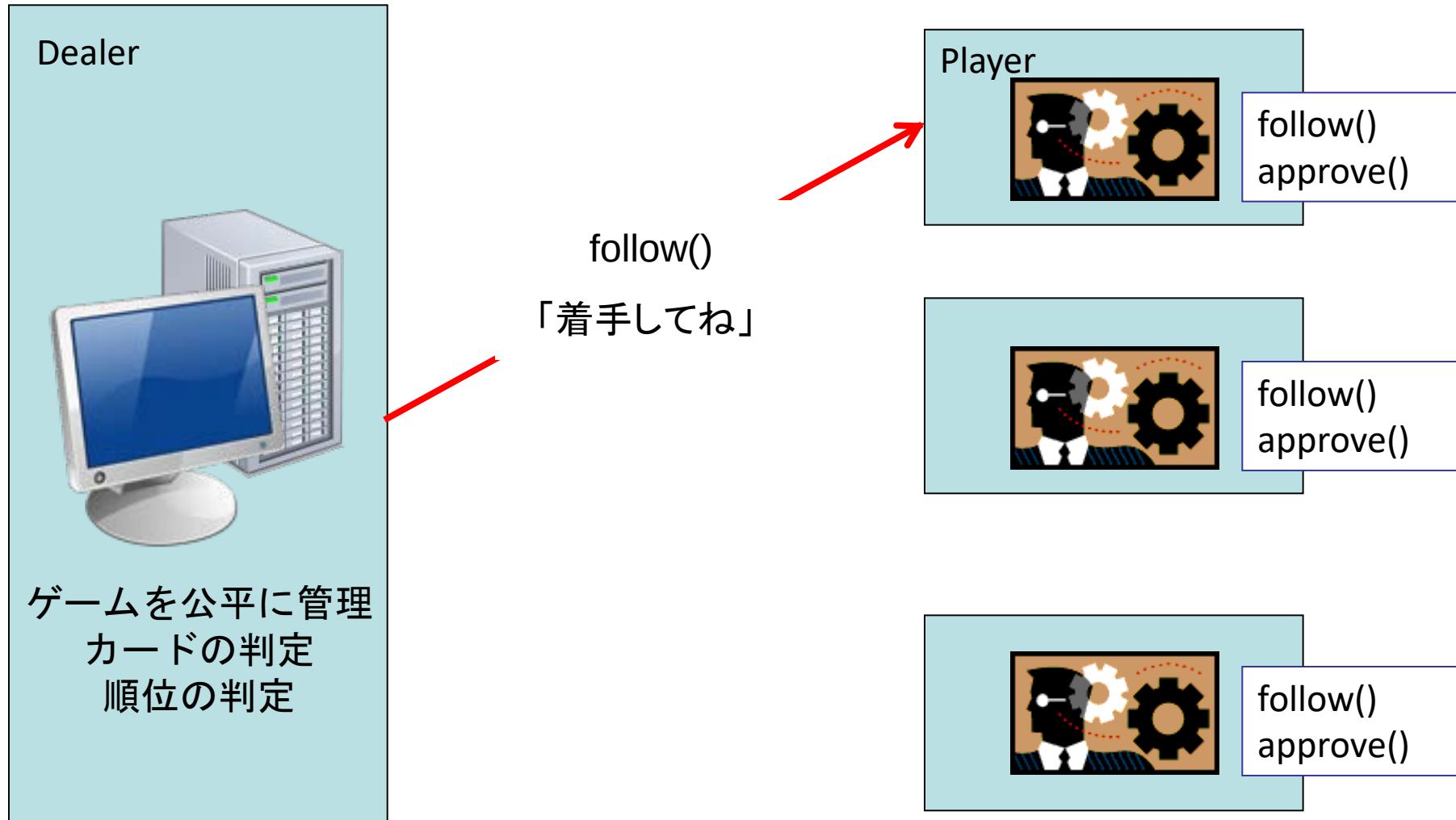
カードの判定(2)

- ゲームとしての判定
 - 空のセット(0枚)や, 通らない(リードよりも弱い)カードが出されると, 「パス」として扱う
 - ・ 通らなかったカードは自分の手持ちに戻る
 - 当然ながら, 出すカードが通るかどうかは自分で判定しないと, 強いプレイヤは作れない

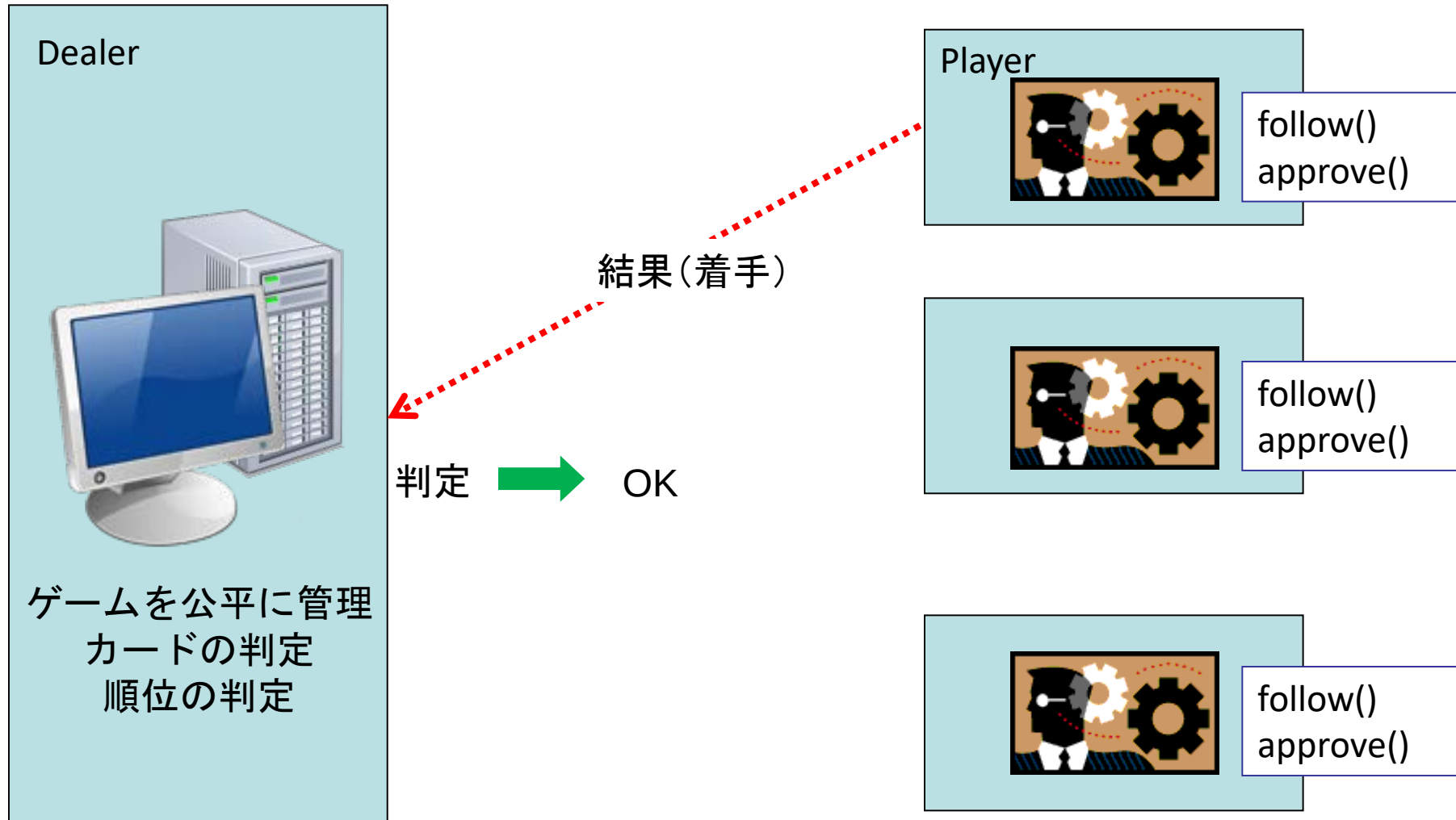
プログラムのイメージ



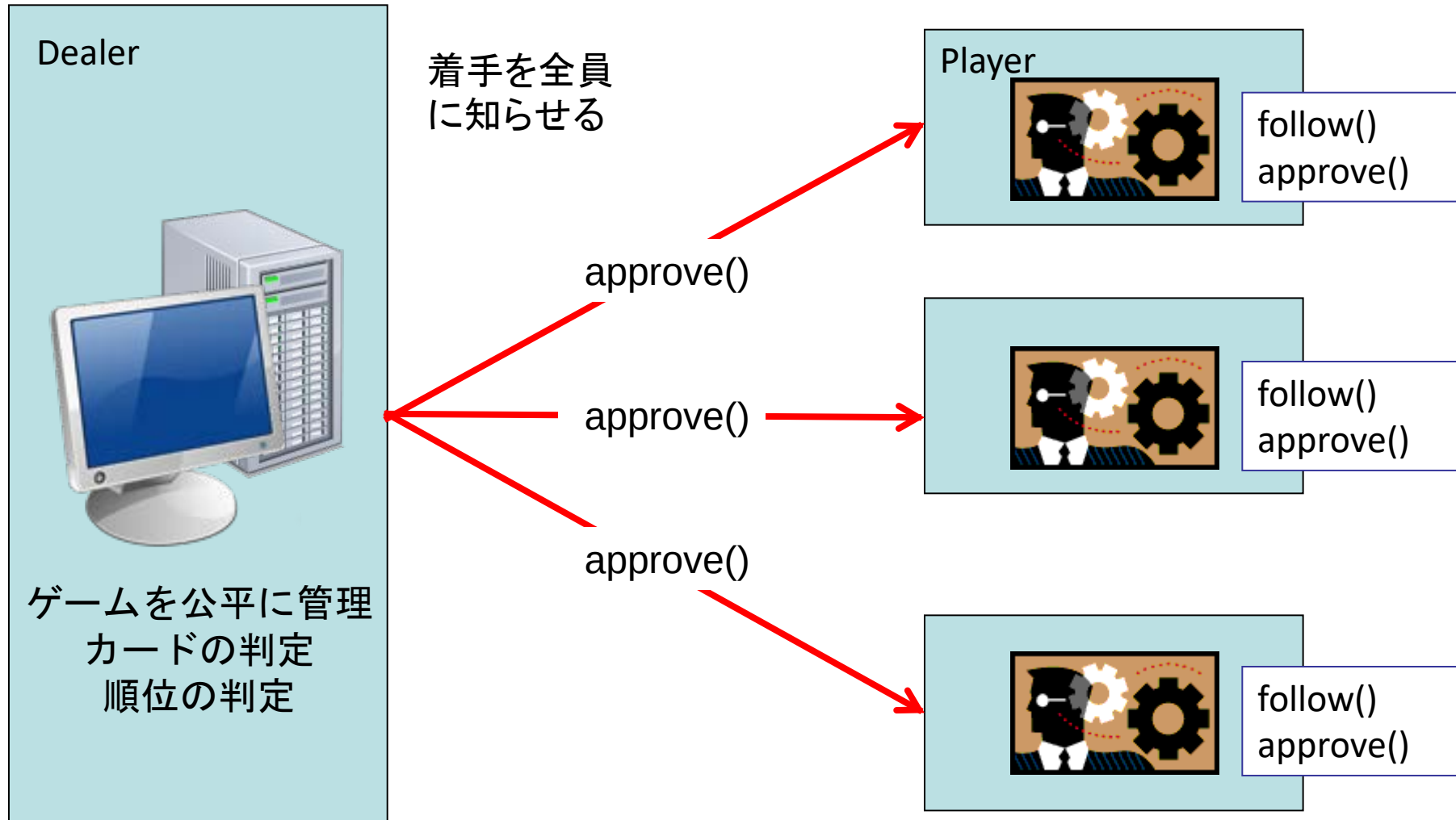
プログラムのイメージ



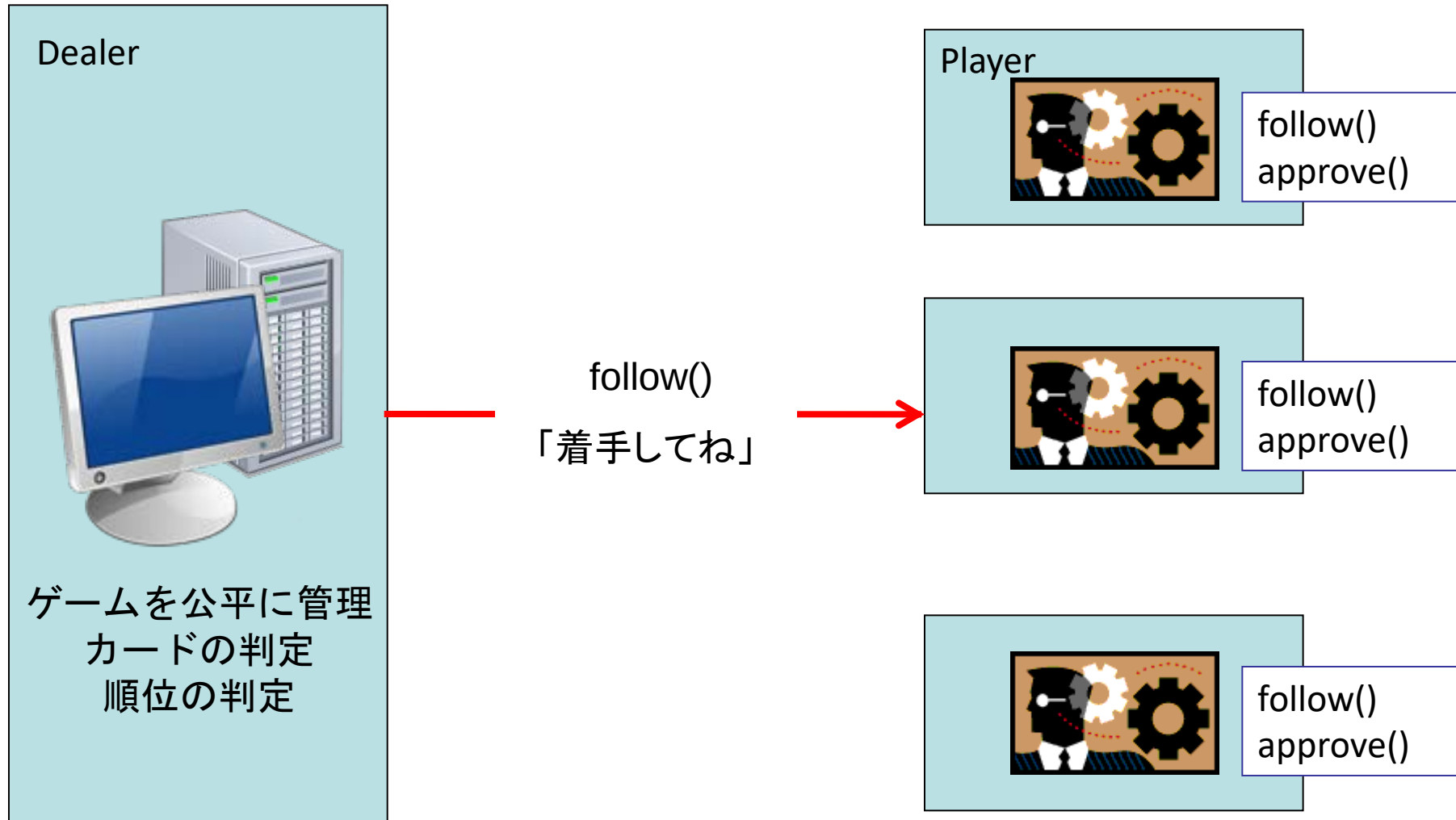
プログラムのイメージ



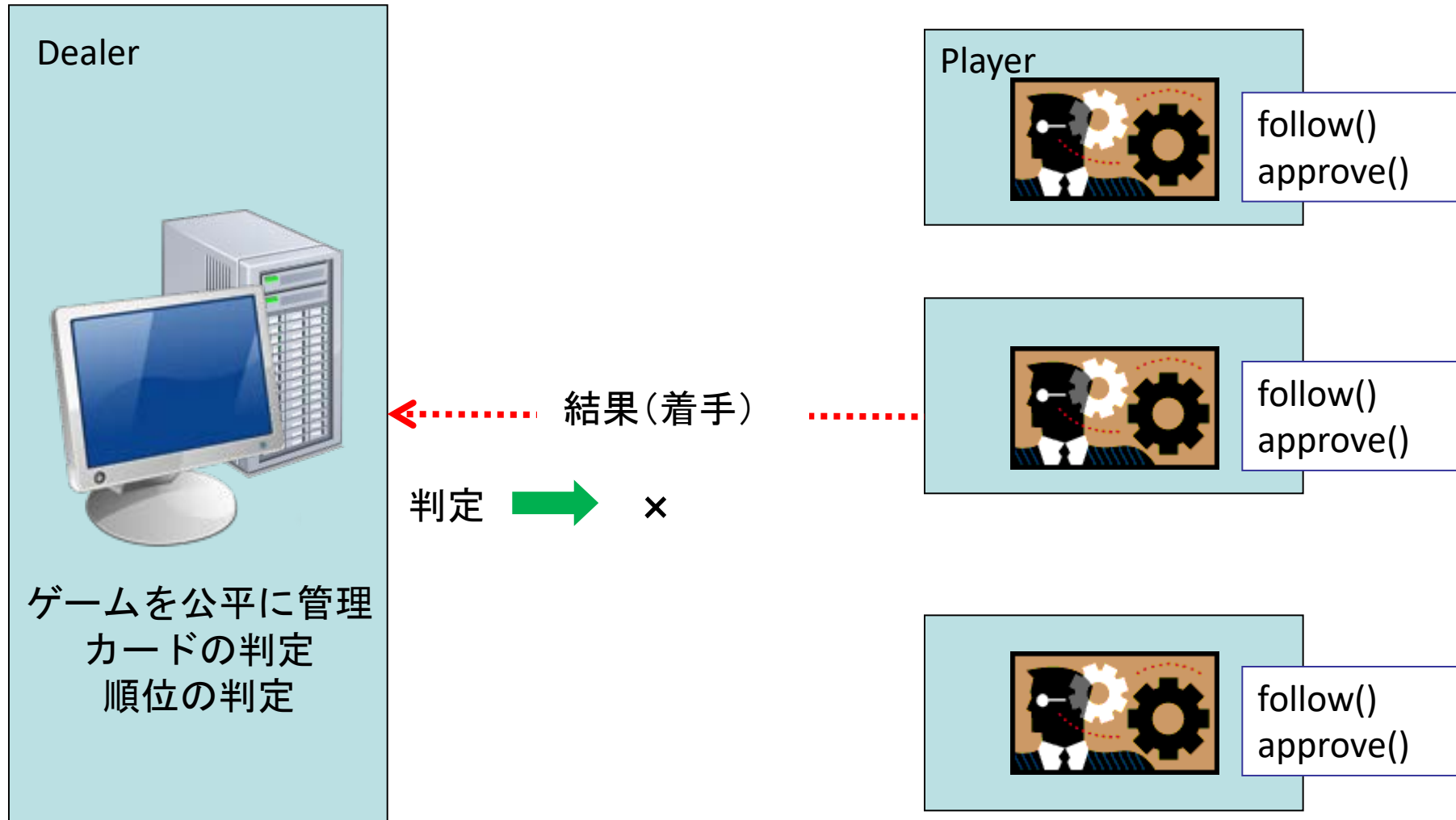
プログラムのイメージ



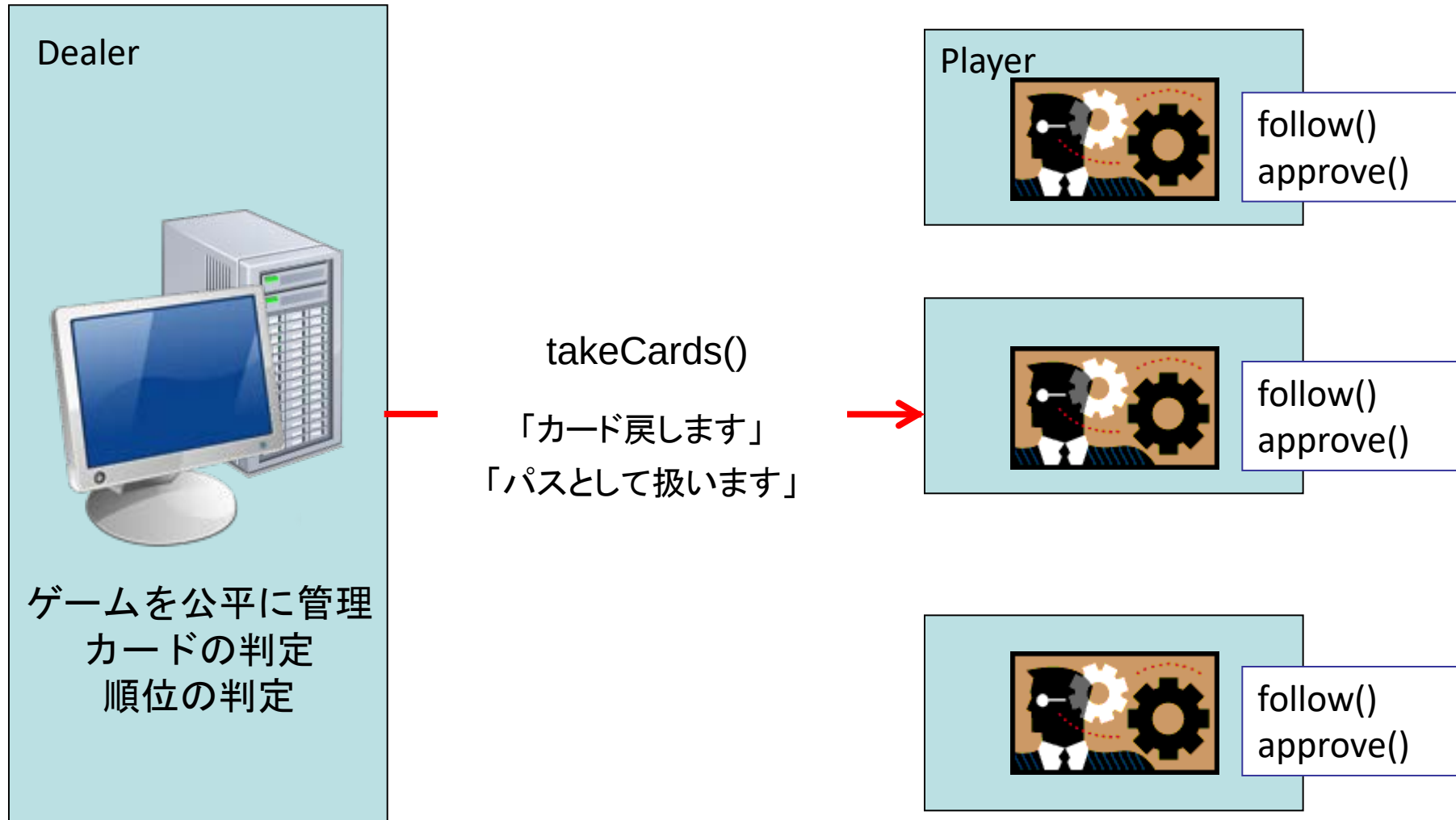
プログラムのイメージ



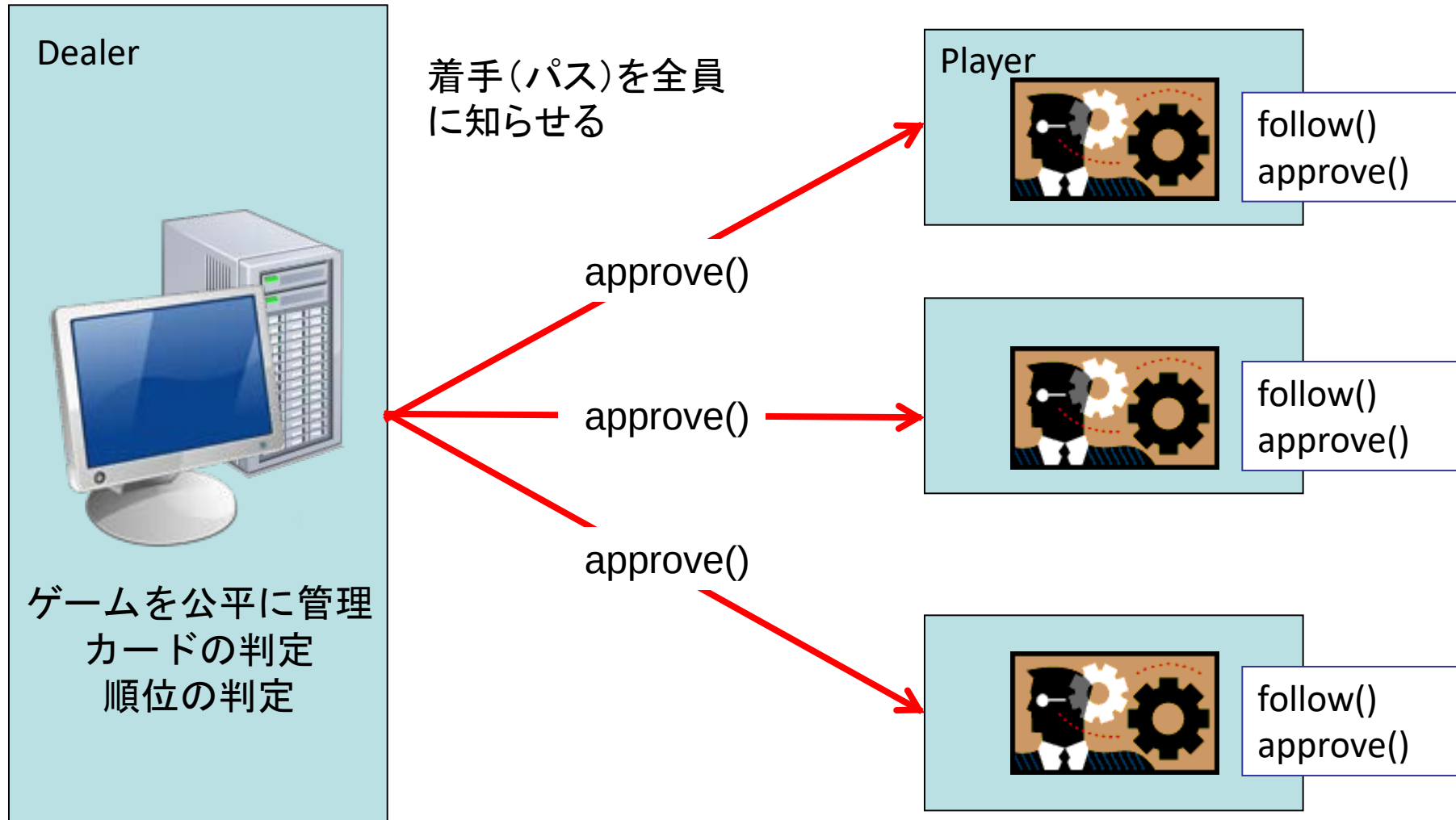
プログラムのイメージ



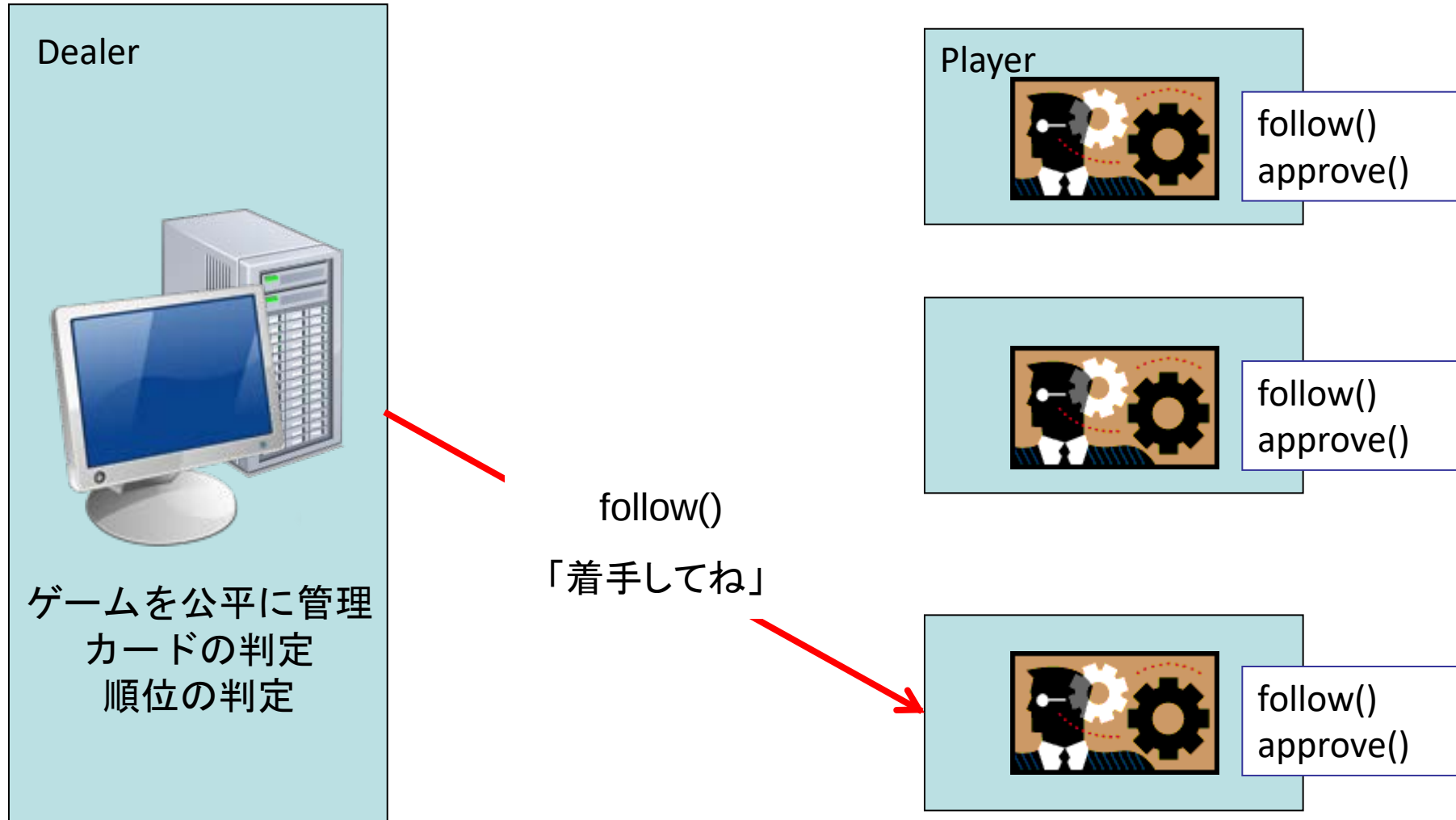
プログラムのイメージ



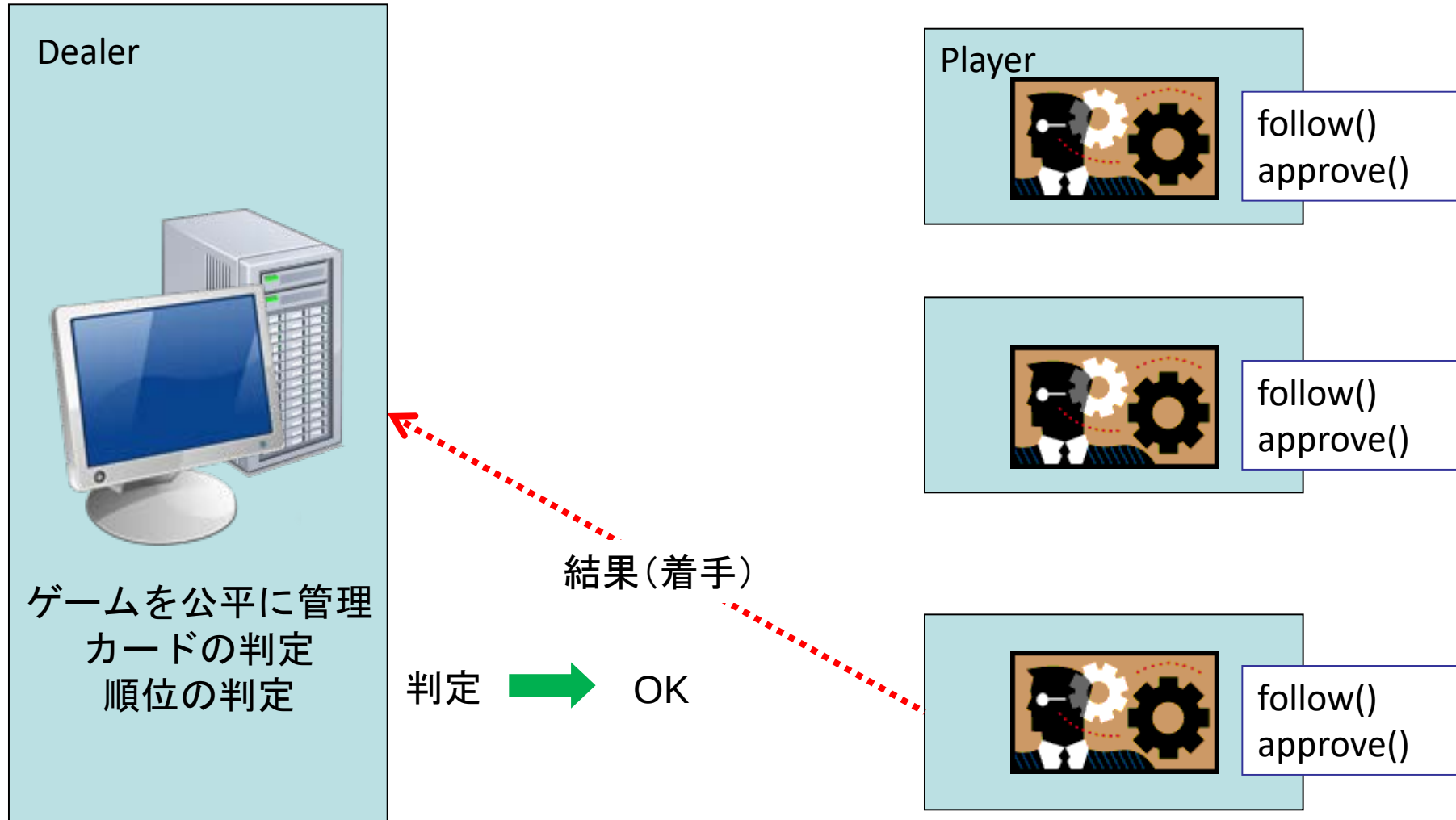
プログラムのイメージ



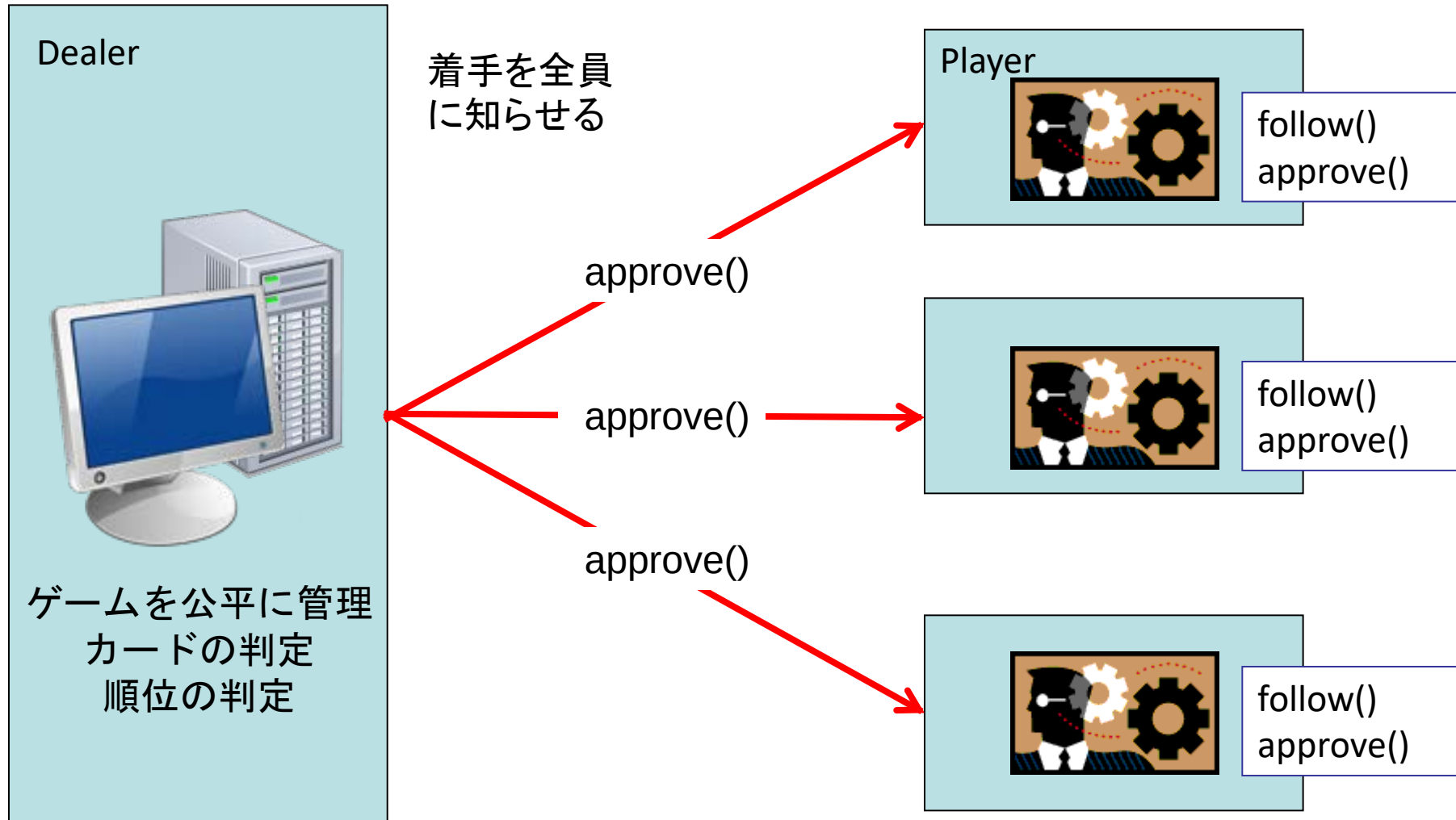
プログラムのイメージ



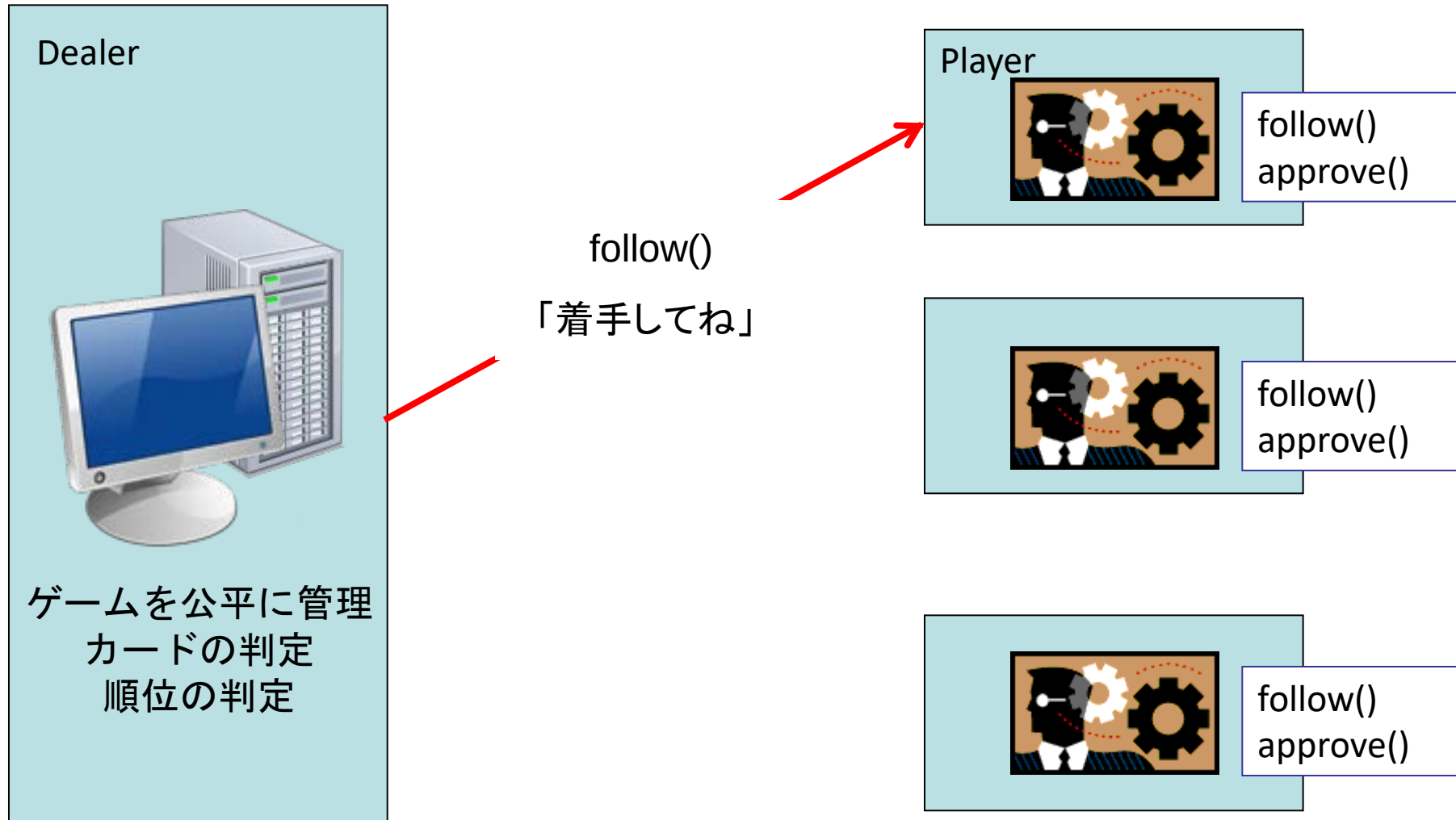
プログラムのイメージ



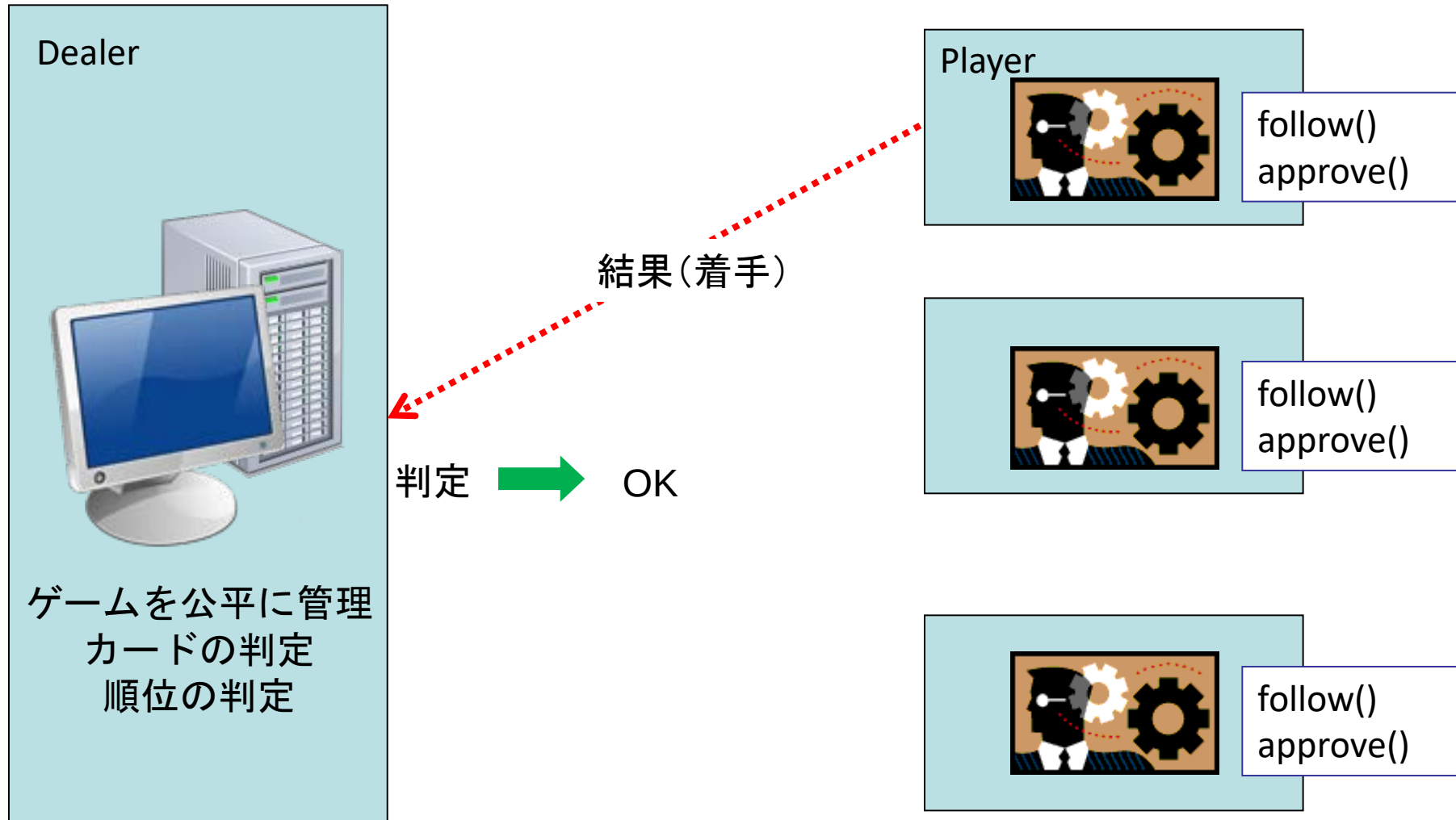
プログラムのイメージ



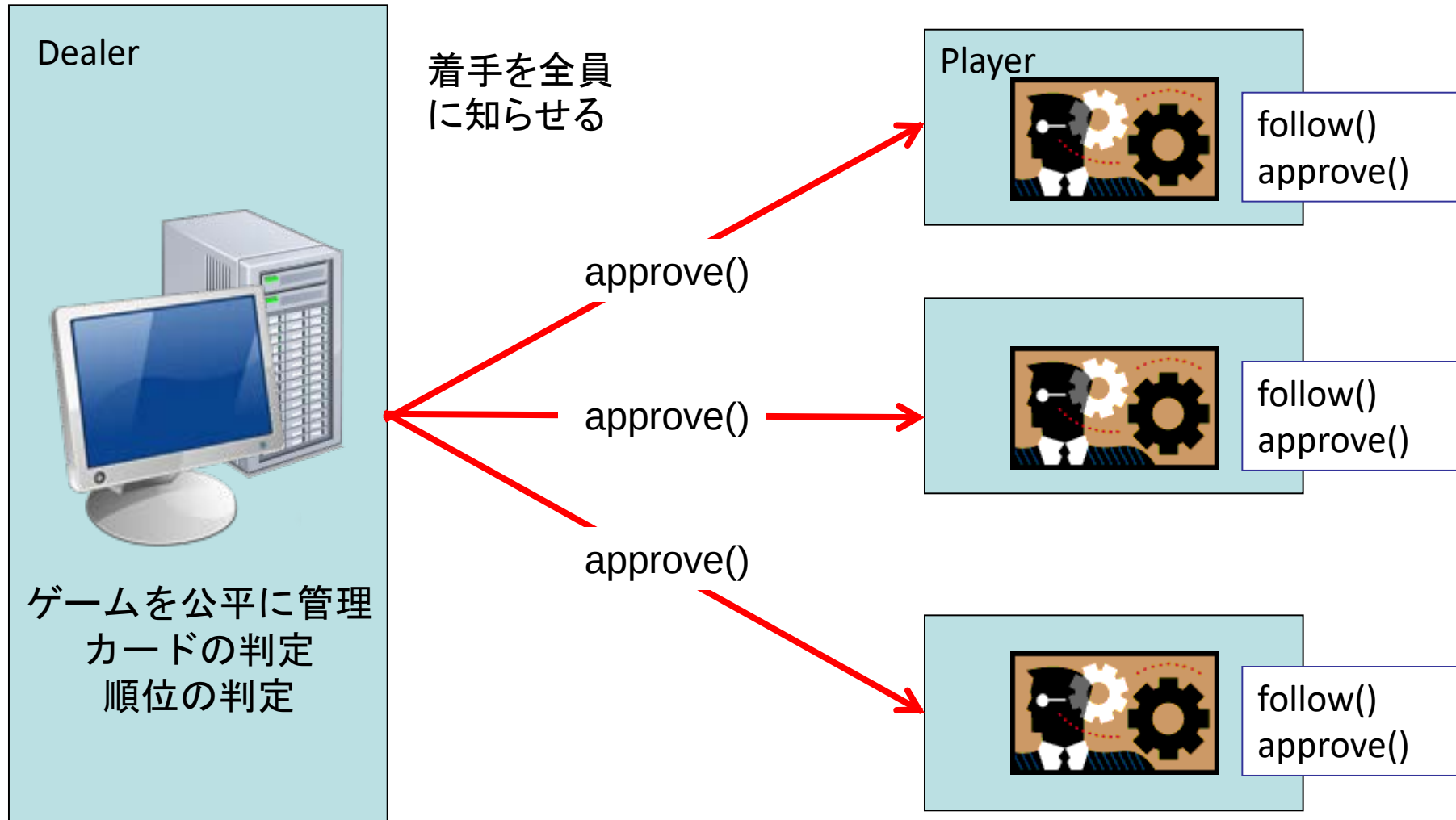
プログラムのイメージ



プログラムのイメージ



プログラムのイメージ



グループでの作業の進め方

- 皆で協力して計画的に
- 大きな流れ(サイクル)
 - プレイヤ作成 → 対戦 → 議論 → 改良 → ...

例えば,

- 最初は 1 人 1 つずつ作成
- 対戦させて問題点を考察
- いくつかのプレイヤを統合して改良
- ...繰り返し最終的に班で 1 つの最強プレイヤを作成

プレイヤーの作り方

- 欲張らず，コツコツと進める
 - 3人寄れば文殊の知恵
 - 少しずつ強くしていく
 - できるだけ弱いカードから出す
 - 選んだカードが場に通るか考えて出す
 - 複数カードを出せるようにする(かなり強力)
- Player クラスを継承してプレイヤクラスを作成
- その中で ready(), follow(), approve() を実装
 - やり方は，SimplePlayer クラスを参考に

複数プレイヤーの実現

● 継承によりサブクラスを作成する

- simplePlayer.{h|cpp} を参考に

```
class SimplePlayer : public Player {  
  
    ...  
  
public:  
  
    SimplePlayer(const char *name = "Simple") : Player(name) { }  
    ~SimplePlayer() { }  
  
    void ready(void);  
    bool follow(const GameStatus &gstat, CardSet &cs);  
    bool approve(const GameStatus &gstat);  
  
    ...  
  
};
```

演習の進め方

- 一人の力に頼らない
 - みんなで分担すること
 - 必ず全員がプログラム作成にたずさわること
 - それが採用されるかどうかは問わない
- 自分たちが作成したプレイヤクラスが、他の班のプログラムと一緒にして動くように注意
 - 班同士の対戦をするため、最終的に提出するソースファイル名とクラス名は以下のとおりとせよ

例えば第1グループならば

- クラス名 : Group1
- ファイル名 : group1.h, group1.cpp

レポート: 提出物 (1)

- グループレポート
 - 班で 1 つ作成して代表者が提出
 - 表紙には, 演習名, 班員の学生番号と名前, 提出日
 - 各班員の役割分担
 - 実装した戦略や作業などを箇条書きする程度でよい
 - ただし, 誰が何をやったかはわかるようにすること
- プログラムリスト
 - 最終的に実装した部分のみでよい
- 実装したアイデア, 採用した理由など
 - 工夫した点, できた部分, できなかった部分, など
- 実行結果
 - 主要な部分・レポートでの説明に必要な部分のみでよい
- 全体の考察とまとめ

レポート: 提出物 (2)

● 個人評価票

- 全員がそれぞれ**個人**で作成して提出
- 自己評価と各班員の評価を記入
- 様式は personal-cpp.docを使用
- 記入後, pdf 化して提出

レポート提出先と締切および評価

- 提出物
 - グループレポート: 班の代表者が提出
 - 個人評価票: 全員が提出
- 提出先
 - Moodle の「後半のレポート提出箱」
- 提出締切
 - 演習最終日の 2 週間後の 17 時
- 班のレポート + 個人評価票 + 大会順位に基づく配点

大会について

- Moodle に group[番号].cpp および .h の提出
 - デバッグ用のコメントなどは出さないようにしておく！
- 最終回の 15:30 (予定)より大会開催
 - 抽選により 3 班(+ ThinkTA1, SimplePlayer)で 1 リーグの 1 次予選
 - 1 次予選 1 位は決勝進出, 2 位は敗者復活戦 2 へ, 3 位は敗者復活戦 1 へ
 - 敗者復活戦 1 (+ ThinkTA1, SimplePlayer) の 1 位が敗者復活戦 2 へ
 - 敗者復活戦 2 (+ ThinkTA1) の 1 位が決勝へ
 - 予選通過の 4 チームと ThinkTA1 の 5 チームで決勝戦
 - 上位 3 位と過去の 1 位チーム(2 つ)でエキシビション
- 順位と得点 (n チームでの対戦の場合)
 - 対戦毎に, 1 位: $n+1$ 点, 2 位: $(n-1)$ 点, 3 位: $(n-2)$ 点, ...
 - 全対戦(0.5~1 万対戦を予定)の得点の合計で最終順位を決定

大会について

